

The Problem Terraform Solves:

We have been creating AWS resources manually (clicking through the AWS Console). Now we want to automate this.

Terraform is a tool that lets us write instructions for creating AWS resources, instead of clicking.

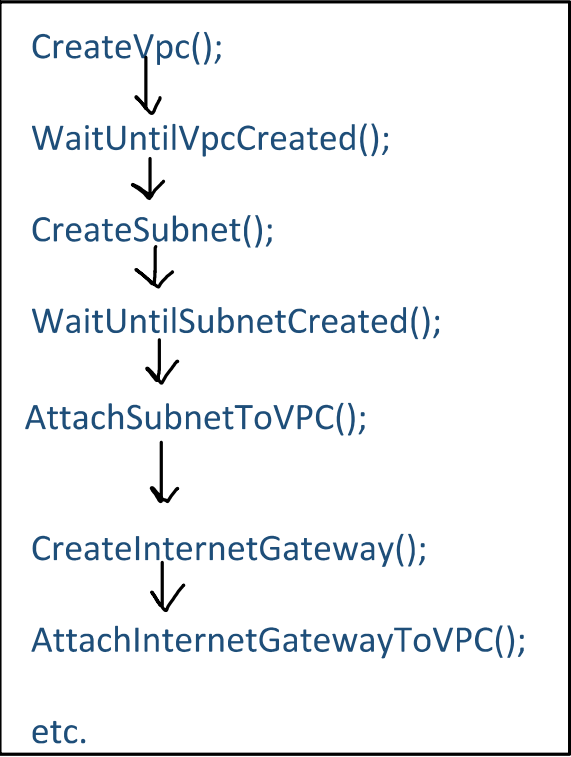
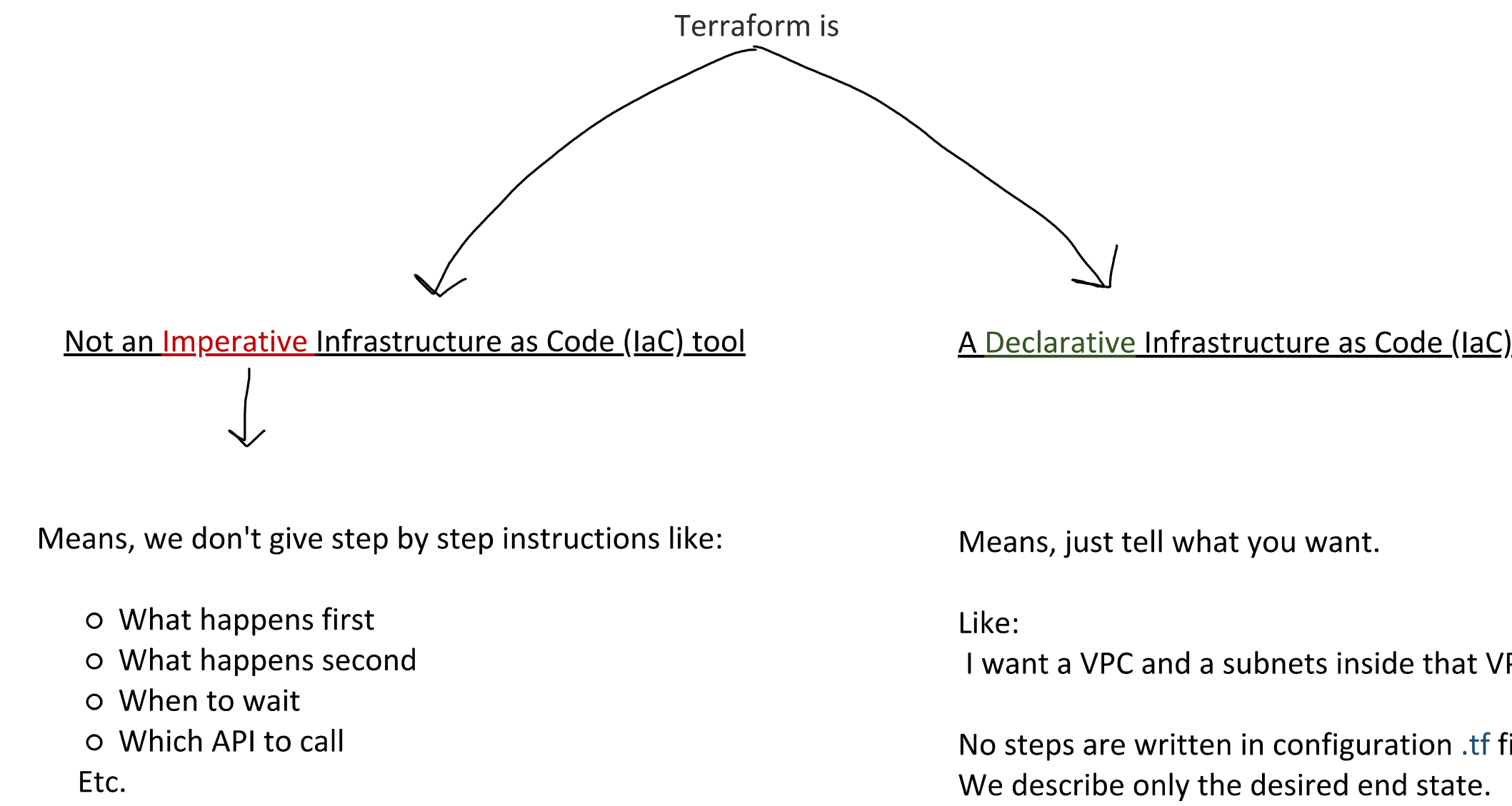
So in simple term, using Terraform:

We can write instructions --> Terraform reads them --> Terraform calls AWS API --> AWS creates resources

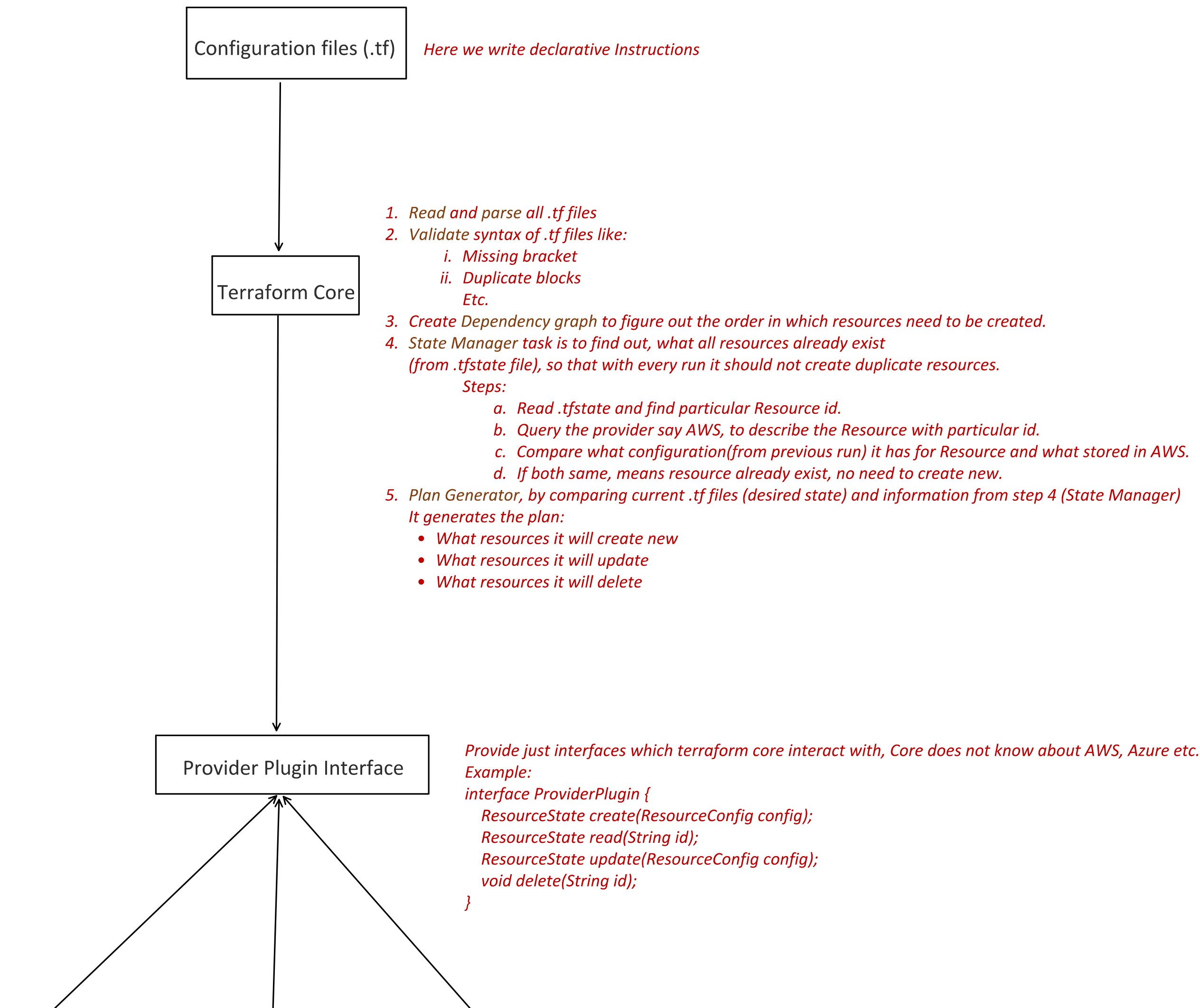
Infrastructure as Code:

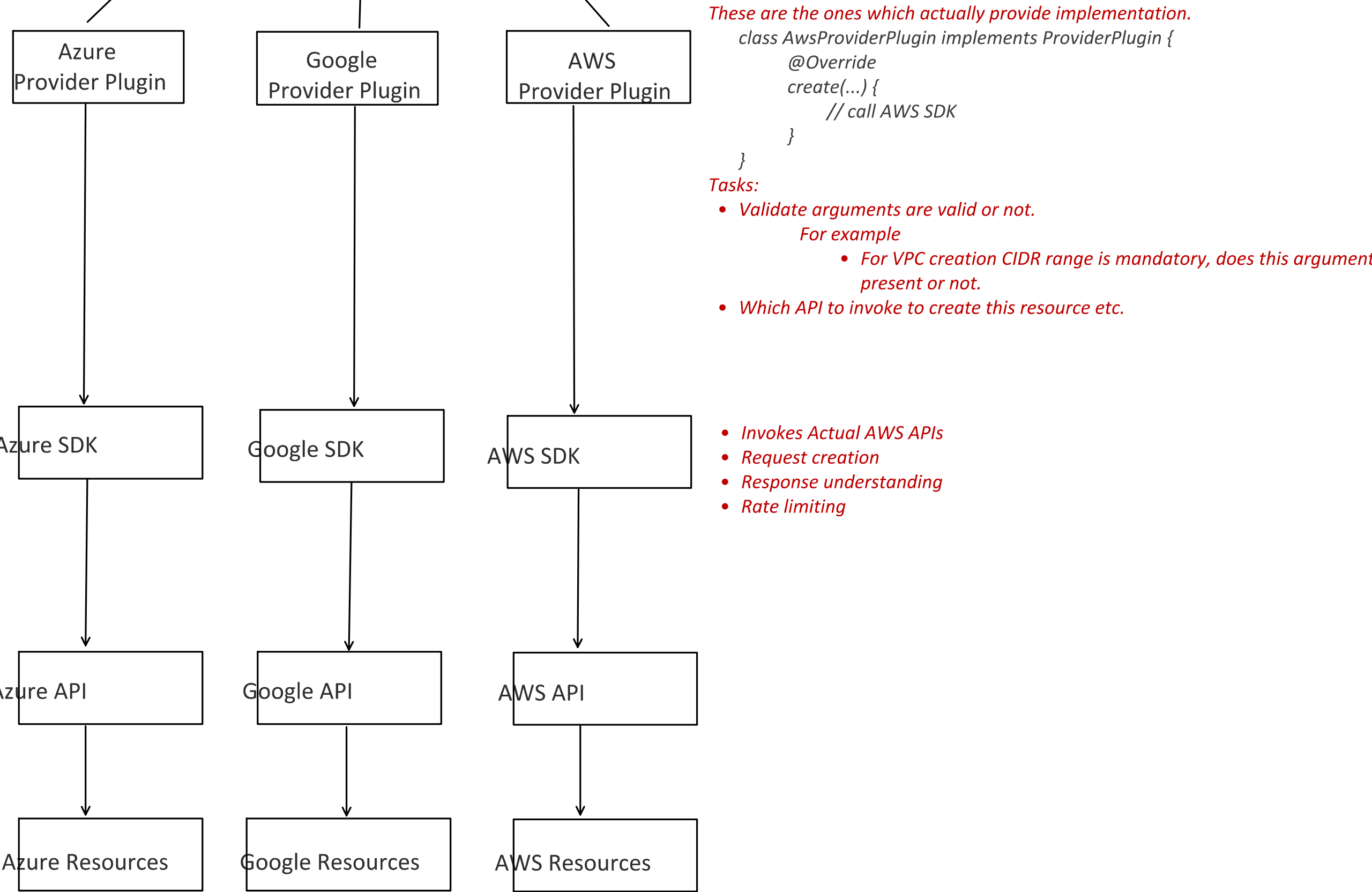
Terraform uses files with the .tf extension. These are plain text files where we define our desired infrastructure configuration.

Since these files are just text, we can store them in Git to maintain version history. This allows us to track what changed, when it changed, who changed it, and why it changed.



Terraform Architecture:



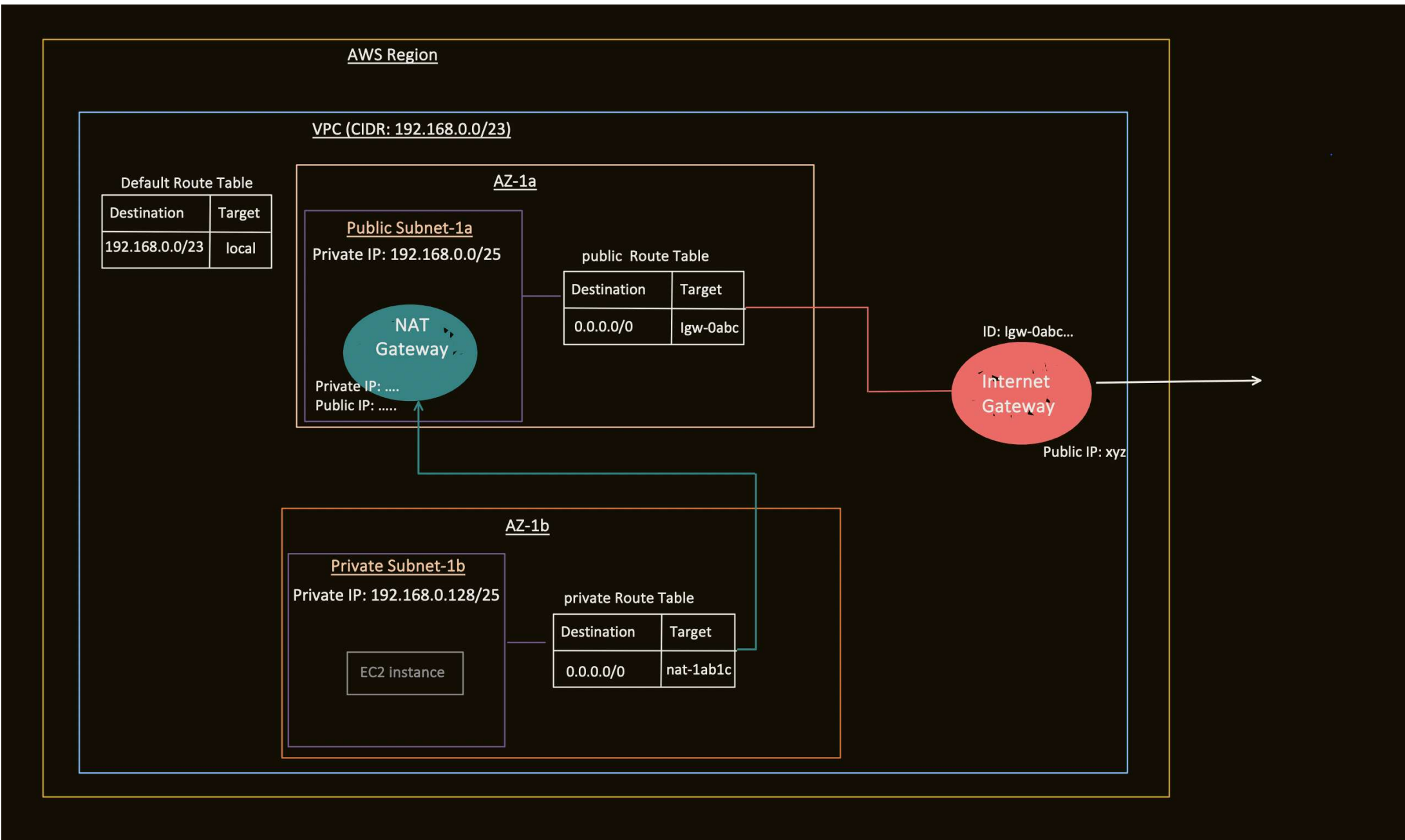


Blocks:

Everything in Terraform is written in blocks. A block is like a container that holds information.

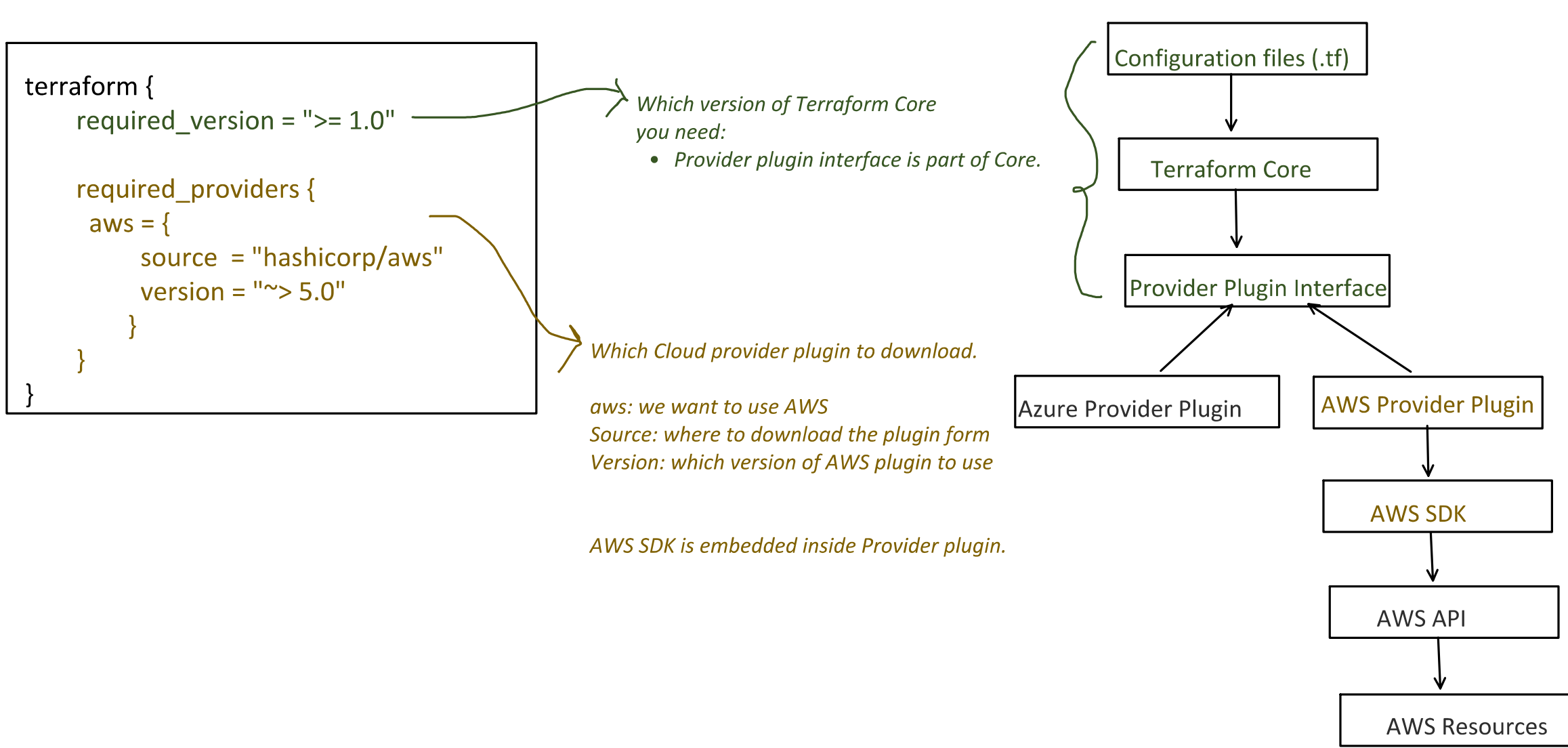
```
block_type "label1" "label2" {  
  key = value  
  key = value  
}
```

Note:
There are different type of blocks, today we will cover the ones which are required to automate the below infrastructure which we set up manually through AWS Console:



Terraform Block:

- It helps to configure Terraform itself.
- It does not create any resources. Jut configure terraform settings.



Provider Block:

- In this block, we can configure the Region in which we want to create the resources.
- But this does not create any AWS resources.

```
provider "aws" {  
  region = "us-east-1"  
}
```

- Provider "aws" ----> it tells terraform to use the AWS provider plugin
- While making connection with AWS, use the given region.

Resource Block:

- Using resource block, we can say: **I want this resource to exist**

```
resource "{resource_which_we_want_to_exist}" "{custom_resource_name}" {  
  {arguments 1} = {value}  
  {argument 2} = {value}  
}
```

Example:

```
resource "aws_vpc" "main" {  
  cidr_block = "192.168.0.0/23"  
}
```

How to know?

- the Resource name:
 - Like it should be "aws_vpc" or something else, how do I know?
How do I know what is the resource name for example for subnet, internet gateway etc. what's the name?
- the Arguments:
 - How do I know, what all arguments I can provide within the resource?
 - Is that argument optional or mandatory, how do I know.
 - Also that argument accepts string or integer etc. How do I know.

The answer is:

Terraform Registry Website: <https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/vpc>

aws_nat_gateway

aws_nat_gateway_eip_association

aws_network_acl

aws_network_acl_association

aws_network_acl_rule

aws_network_interface

aws_network_interface_attachment

aws_network_interface_permission

aws_network_interface_sg_attachment

aws_route

aws_route_table

aws_route_table_association

aws_security_group

aws_security_group_rule

aws_subnet

aws_vpc

aws_vpc_block_public_access_exclusion

aws_vpc_block_public_access_options

Select the resource

You will find all the details of this selected resource

1. List of Arguments supported by this resource:
ex:

- region - (Optional) Region where this resource will be managed.
- cidr_block - (Optional) The IPv4 CIDR block for the VPC.
 - .
 - .
 - .

2. Attribute (the output we can get from this resource)

- arn - Amazon Resource Name (ARN) of VPC
- id - The ID of the VPC
 - .
 - .
 - .

Very important, resource does not say "CREATE this resource".

It just say, This resource should exist.

Now terraform will first check, if resource already exist, it will not create, else it will create.

If we want to delete the resource:

Say previously we had:

```
resource "aws_vpc" "main" {  
  }  
resource "aws_subnet" "public" {  
}
```

Now changed to

```
resource "aws_vpc" "main" {  
}
```

"STATE MANAGER"
compares the current .tfstate with existing .tf and find out the

difference that, "aws_subnet" resource is not required.
So it will create the plan accordingly.

Variable Block:

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_subnet" "public" {  
  region= "us-east-1"  
}  
  
resource "aws_subnet" "private" {  
  region = "us-east-1"  
}
```

All 3 blocks are using 1 value "us-east-1"

Now lets say, I need to change this region value.
Then I need to update all 3 places.

Instead:

```
variable "aws_region" {  
  description = "region where resources will be created"  
  type = string  
  default = "us-east-1"  
}
```

```
provider "aws" {  
  region = var.aws_region  
}  
  
resource "aws_subnet" "public" {  
  region= var.aws_region  
}  
  
resource "aws_subnet" "private" {  
  region = var.aws_region  
}
```

Now, any change in region, we need to change only at 1 place.

Some Common Types:

Type	Example
string	"us-east-1"
number	20
bool	false / true
list(Type)	list(string) ["a", "b"]
set(Type)	set(string) ["a", "b", "a"] it will remove duplicates but order is not maintained
map(Type)	map(string) { name1 = "value1" name2 = "value2" }